



Aplicaciones e Industria de IoT

Semana 3 — Gestión de Datos, Analítica y ML en IoT

KOICA

HGU HANDONG GLOBAL
UNIVERSITY



UNA

ÍNDICE

- 1. El Dato en IoT: Tipos, Ciclo de Vida y Series Temporales**
- 2. Pipeline de Datos IoT**
- 3. Analítica y Machine Learning Aplicado a IoT**
- 4. Casos de Éxito: IoT en Acción**
- 5. Laboratorio PBL: Pipeline Completo de Cadena de Frío**
- 6. Resumen, Bibliografía y Próximos Pasos**



Repaso de la Semana 2 y Objetivos de la Semana 3



Lo que aprendimos en la Semana 2

- Protocolos IoT clave: MQTT (Pub/Sub), HTTP/REST, CoAP.
- Tecnologías de red: Wi-Fi, BLE, LoRaWAN, NB-IoT, 5G.
- Plataformas IoT: ThingsBoard, AWS IoT, Azure IoT Hub.
- Metodología de 6 pasos para diseñar arquitecturas IoT.
- LAB: Comunicación MQTT real con ESP32 y ThingsBoard.
- Diseño de diagrama de arquitectura para proyecto integrador.



Objetivos de la Semana 3

- Comprender el ciclo de vida del dato IoT y tipos de datos.
- Diseñar un pipeline de datos: ingesta → almacenamiento → procesamiento → visualización.
- Aplicar técnicas de analítica descriptiva sobre series temporales.
- Introducir conceptos de Machine Learning para IoT.
- Implementar generación de datos con ESP32 + MQTT en Wokwi.
- Analizar datos con Python (pandas, matplotlib) en Google Colab.



Conexión con la Semana 2: Hoy nos enfocamos en qué hacemos con los datos una vez que llegan a la plataforma

4

Aplicación

(Semanas 4-8)

3

Plataforma

HOY: Plataformas IoT

2

Red

(Semana 2)

1

Dispositivo

(Semana 1: ESP32 + sensores)

1. EL DATO EN IoT: TIPOS, CICLO DE VIDA Y SERIES TEMPORALES

¿Por Qué Importan los Datos en IoT?

Los dispositivos IoT generan volúmenes masivos de datos. Un sensor industrial típico produce miles de lecturas por hora. Sin una estrategia de gestión de datos, estos datos carecen de valor. La cadena Dato → Información → Acción es lo que transforma sensores en soluciones.



Datos Clave del Ecosistema IoT

80 ZB

Datos IoT estimados
para 2025 (IDC)

95%

De datos IoT que
nunca se analizan

6×

ROI promedio en proyectos
con analítica aplicada

Fuentes: IDC Global DataSphere Forecast, McKinsey IoT Value Report

Tipos de Datos en IoT – Clasificación Fundamental



Los 4 Tipos de Datos que Genera un Sistema IoT



Series Temporales

El dato más frecuente en IoT

Mediciones continuas indexadas por tiempo. Ejemplo: temperatura cada 5 s, consumo eléctrico horario, vibración de un motor. Requieren bases de datos especializadas (InfluxDB, TimescaleDB).



Eventos Discretos

Ocurrencias puntuales

Sucesos con marca de tiempo: apertura de puerta, alarma de presión, detección de movimiento, cambio de estado ON/OFF. Se almacenan como logs y disparan reglas.



Datos de Estado

Parámetros que cambian poco

Valor actual de configuración o condición: firmware, modo de operación, nivel de batería, ubicación GPS. Se consultan bajo demanda, no se grafican como series.



Multimedia / Binarios

Contenido pesado

Imágenes, audio o video de cámaras o micrófonos IoT. Inspección visual en fábricas, vigilancia urbana. Requieren procesamiento en el borde (edge) por su volumen.

Series Temporales – El Dato Central de IoT

📄 ¿Qué es una Serie Temporal?

Una serie temporal es una secuencia de valores numéricos registrados a intervalos regulares o irregulares de tiempo. En IoT, cada lectura de un sensor genera un punto en la serie.

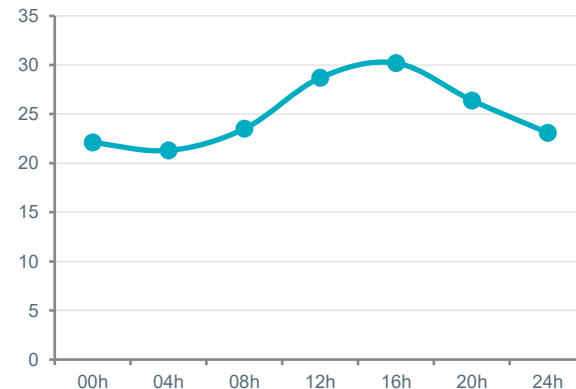
Componentes clave:

- Tendencia (trend): dirección general a largo plazo.
- Estacionalidad: patrones que se repiten en ciclos (diarios, semanales).
- Ruido: variaciones aleatorias inherentes a la medición.
- Anomalías: valores atípicos que pueden indicar fallas.

Desafíos principales:

Volumen masivo, velocidad de ingesta, calidad variable, sincronización entre dispositivos y políticas de retención (TTL).

📊 Ejemplo: Temperatura 24h



*Patrón diario típico con pico a las 16h.
La estacionalidad diaria es visible.*

2. PIPELINE DE DATOS IoT

Pipeline de Datos IoT – Visión General

Un pipeline de datos define el flujo completo desde que el sensor captura una medición hasta que se genera una acción. Diseñar un pipeline robusto es la clave para extraer valor real de los datos IoT.



Las 5 Etapas del Pipeline

INGESTA

Recibir, validar,
enriquecer



ALMACENAR

Persistir en BD
de series temporales



PROCESAR

Limpiar, transformar,
agregar



ANALIZAR

Estadísticas, ML,
detectar patrones



VISUALIZAR

Dashboards, alertas,
reportes



Procesamiento Batch

Datos acumulados en lotes programados (cada hora/día). Ideal para reportes históricos, entrenamiento de modelos ML. Herramientas: Apache Spark, scripts ETL, cron jobs.



Procesamiento en Tiempo Real

Cada dato procesado al llegar (stream). Esencial para alertas inmediatas, control de procesos, detección de anomalías en vivo. Herramientas: Kafka Streams, Node-RED, AWS IoT Rules.

Etapa 1: Ingesta de Datos — Protocolos y Validación

La ingesta es el punto de entrada de los datos al pipeline. Define cómo los dispositivos envían información a la plataforma. Incluye validación de formato, enriquecimiento con metadatos (ID, ubicación, timestamp) y gestión de colas para absorber picos de tráfico.



Tabla Comparativa — ¿Cuándo Usar Cada Protocolo de Ingesta?

Protocolo	Modelo	Formato	Caso de Uso IoT
MQTT	Publish / Subscribe	JSON, binario	Telemetría masiva de sensores
HTTP/REST	Request / Response	JSON, XML	APIs, configuración, dashboards
WebSocket	Bidireccional persistente	JSON, binario	Dashboards en vivo, control remoto
Kafka / AMQP	Cola de mensajes	Avro, JSON	Ingesta empresarial de alto volumen

Etapa 2: Almacenamiento — Bases de Datos para IoT

¿Qué Base de Datos Elegir para Datos IoT?

InfluxDB

Series temporales

Optimizada para datos con timestamp. Consultas rápidas sobre rangos temporales. Retención (TTL) automática configurable. Lenguaje InfluxQL o Flux.

TimescaleDB

Series temporales (PostgreSQL)

Combina SQL completo con rendimiento de series temporales. Ideal para integrar datos IoT con datos relacionales existentes en una misma BD.

MongoDB

Documental (NoSQL)

Esquema flexible para datos heterogéneos. Escalamiento horizontal. Buena para metadatos, eventos y configuraciones de dispositivos.

Apache Cassandra

Columnar distribuida

Alta disponibilidad y tolerancia a fallos. Ideal para escrituras masivas distribuidas geográficamente. Usado por Netflix, Apple.

Edge Computing vs. Cloud Computing en IoT

El procesamiento en el borde (edge) ejecuta lógica en el dispositivo/gateway. La nube ofrece cómputo ilimitado para análisis complejos. La tendencia actual son arquitecturas híbridas que combinan ambos según las necesidades de cada flujo.

Tabla Comparativa — ¿Cuándo Usar Edge y Cuándo Cloud?

Criterio	Edge	Cloud
Latencia	Muy baja (ms)	Media-alta (s a min)
Capacidad de cómputo	Limitada	Escalable, potente
Conectividad requerida	Funciona sin conexión	Requiere conexión estable
Ancho de banda	Bajo (datos filtrados)	Alto (datos brutos)
Caso de uso ideal	Alertas inmediatas, control local	Analítica profunda, entrenamiento ML

Etapa 4: Visualización – Herramientas y Dashboards



Herramientas de Visualización IoT – Comparativa

Grafana

Dashboard de código abierto. Conecta con InfluxDB, Prometheus, SQL. Alertas configurables. Estándar de la industria para monitoreo de infraestructura y IoT.

ThingsBoard CE

Plataforma IoT completa con dashboards integrados. Soporta MQTT, HTTP, CoAP nativamente. Ideal para el curso: la usamos desde la Semana 2.

Google Sheets / Looker

Gratuito y accesible para prototipos rápidos. Gráficos básicos con actualización semi-automatizada mediante Apps Script.

Node-RED Dashboard

Interfaz visual basada en flujos (drag & drop). Prototipos rápidos con gauges, gráficos y botones interactivos conectados a MQTT.

Ejercicio – Generar Datos con ESP32 + MQTT en Wokwi



EJERCICIO EN CLASE

 **Objetivo:** Publicar datos del DHT22 vía MQTT desde el ESP32 simulado en Wokwi



Instrucciones Paso a Paso

1. Abrir wokwi.com → ESP32 → nuevo proyecto.
2. Agregar DHT22: VCC→3V3, GND→GND, DATA→GPIO4.
3. En **Libraries** agregar:
 - DHT sensor library
 - PubSubClient (librería MQTT para Arduino)
4. Copiar el código de la siguiente diapositiva.
5. Ejecutar simulación y abrir Serial Monitor.
6. Verificar que aparezca: "Conectado al broker MQTT" y los mensajes publicados.
7. Abrir MQTTX (cliente) y suscribirse al topic para ver los datos en tiempo real.



¿Qué Van a Aprender?

- Cómo generar datos IoT estructurados (JSON) con timestamp.
- La diferencia entre `Serial.print` (local) y `mqtt.publish` (remoto).
- Cómo un dato del sensor alimenta un pipeline real.
- Verificar datos con un cliente MQTT externo.



Datos del Broker Público

Servidor: `broker.hivemq.com`
Puerto: 1883 (sin cifrado)
No requiere usuario ni contraseña
Topic sugerido: `iot-fpuna/sem3/TU-NOMBRE/temp`

Ejercicio · Código ESP32 + MQTT en Wokwi



Código Completo — Copiar en sketch.ino de Wokwi

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#define DHTPIN 4
#define DHTTYPE DHT22
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* topic = "iot-fpuna/sem3/temp";
const char* sensor_id = "ESP32-PLANTA-001";
WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;

void setup() {
  Serial.begin(115200); dht.begin();
  WiFi.begin(ssid, password);
  while(WiFi.status() != WL_CONNECTED){delay(500);
    Serial.print(".");}
  Serial.println("\nWi-Fi OK");
  mqtt.setServer(mqtt_server, mqtt_port);
}

void reconectar() {
  while(!mqtt.connected()){
    if(mqtt.connect(sensor_id))
      Serial.println("MQTT conectado");
    else { delay(3000); }
  }
}

void loop() {
  if(!mqtt.connected()) reconectar();
  mqtt.loop();
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if(isnan(t)){delay(2000); return;}
  seq++;
  char json[200];
  snprintf(json, 200,
    "{\"sensor_id\":\"%s\", \"temp\":%.1f, \"hum\":%.1f, \"seq\":%d}",
    sensor_id, t, h, seq);
  mqtt.publish(topic, json);
  Serial.println(json);
  delay(5000);
}
```

Notas Clave

Wokwi-GUEST

Red WiFi virtual de Wokwi. Sin password.

broker.hivemq.com

Broker público gratuito. Sin autenticación. Solo para pruebas.

Formato JSON:

```
{"sensor_id", "temp", "hum", "seq"}
```

Cada campo mapeará a una columna en el análisis con Python.

Verificar con MQTTX:

1. Conectar a broker.hivemq.com
2. Suscribirse a iot-fpuna/sem3/temp
3. Ver los JSON llegando cada 5 s

3. ANALÍTICA Y MACHINE LEARNING APLICADO A IoT

Niveles de Analítica en IoT – De Descriptiva a Prescriptiva



Los 4 Niveles de Analítica

1

Descriptiva

¿Qué pasó?

Estadísticas, promedios, dashboards históricos. Ej: temperatura media del motor en los últimos 7 días.

2

Diagnóstica

¿Por qué pasó?

Correlaciones, análisis de causa raíz. Ej: picos de temperatura coinciden con horarios de carga máxima.

3

Predictiva

¿Qué pasará?

Modelos estadísticos y ML para anticipar eventos. Ej: predicción de falla del motor en las próximas 48 horas.

4

Prescriptiva

¿Qué hacer?

Recomendaciones y acciones automáticas. Ej: reprogramar mantenimiento y reducir carga del equipo.

Machine Learning Aplicado a IoT – Algoritmos Clave

El aprendizaje automático (ML) permite a los sistemas IoT ir más allá de reglas estáticas: los algoritmos aprenden patrones de los datos históricos para predecir comportamientos futuros, detectar anomalías y optimizar operaciones de forma autónoma.



Algoritmos más Usados en IoT Industrial

Regresión Lineal

Supervisado

Predecir valores continuos: temperatura futura, consumo energético, desgaste de equipo.

Random Forest

Supervisado

Clasificar estados (normal/falla). Robusto, interpretable. Usado en mantenimiento predictivo.

K-Means Clustering

No supervisado

Agrupar dispositivos por comportamiento. Segmentar patrones de uso sin etiquetas previas.

Redes LSTM

Deep Learning

Predicción de series temporales complejas. Mantenimiento predictivo avanzado con memoria a largo plazo.

Detección de Anomalías y Mantenimiento Predictivo



Métodos de Detección de Anomalías en Series Temporales

Umbral estático

Alerta si valor $> X$. Fácil pero no adaptativo. Genera falsos positivos en contextos variables.

Z-Score

Mide desviaciones estándar respecto a la media. Efectivo para distribuciones normales. Umbral: $|z| > 3$.

Media móvil $\pm$ banda

Promedio + desviación sobre ventana temporal. Se adapta a tendencias locales. Muy usado en IoT.

Isolation Forest (ML)

Algoritmo no supervisado que aísla puntos atípicos. No requiere etiquetas. Efectivo multidimensional.

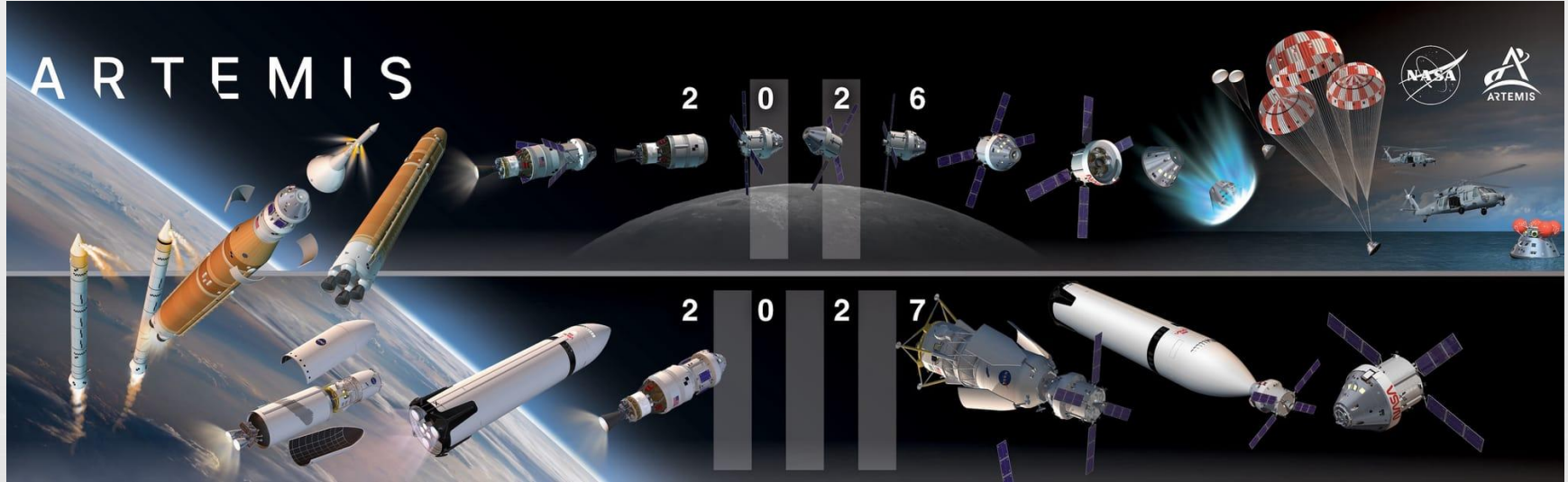


Mantenimiento Predictivo — Beneficios Comprobados

El mantenimiento predictivo usa datos de sensores + modelos ML para predecir fallas antes de que ocurran. Reemplaza el mantenimiento periódico por uno basado en la condición real del activo. Beneficios: reducción de paradas no planificadas 30-50%, extensión de vida útil 20-40%, ahorro en costos 10-25%.

4. CASOS DE ÉXITO: IoT EN ACCIÓN

Caso de Éxito: Programa Artemis de la NASA



Caso de Éxito: Programa Artemis de la NASA

 El programa Artemis busca el regreso del ser humano a la Luna. El cohete SLS y la cápsula Orión integran miles de sensores que generan datos críticos en tiempo real. Su pipeline de telemetría ilustra todos los conceptos de esta semana: ingesta en tiempo real, procesamiento de series temporales, detección de anomalías y toma de decisiones automatizada en milisegundos.



Pipeline de Telemetría Artemis — Del Sensor a la Decisión

Sensores embarcados

Temperatura, presión, vibración, acelerometría, flujo de propelente, voltaje, estado estructural. Más de 1.000 sensores activos.

Transmisión (TDRS)

Red de satélites Tracking and Data Relay Satellite. Comunicación continua Tierra-espacio con ancho de banda dedicado.

Centro de control (MCC)

Mission Control Center en Houston recibe 22 TB de datos de telemetría por lanzamiento. Dashboards especializados por subsistema.

Análisis automatizado

Algoritmos de detección de anomalías sobre datos en streaming. Umbrales dinámicos por fase de vuelo (lanzamiento, órbita, reingreso).

Decisión y comando

Alertas al director de vuelo. Capacidad de enviar comandos correctivos al vehículo. Latencia máxima aceptable: <1 segundo.

Caso de Éxito: Robots Repartidores Autónomos en EE.UU.

 Empresas como Starship Technologies, Kiwibot y Nuro operan flotas de robots repartidores autónomos en ciudades y campus universitarios de EE.UU. Cada robot es un nodo IoT móvil con 6-12 cámaras, LiDAR, GPS RTK, IMU y sensores de temperatura. Generan ~500 MB/hora de datos combinados. Su arquitectura híbrida edge + cloud es un ejemplo perfecto de pipeline de datos IoT aplicado.



 La combinación edge + cloud permite que el robot tome decisiones en ms localmente y entrene modelos complejos centralmente

Procesamiento Edge (en el robot)

- Navegación autónoma en tiempo real (decisiones en ms).
- Detección y evitación de obstáculos con LiDAR + cámaras.
- Fusión de sensores (sensor fusion) a bordo del robot.
- Monitoreo de batería y gestión inteligente de energía.
- Registro local de eventos para diagnóstico posterior.

Procesamiento Cloud (centro de datos)

- Agregación de telemetría de toda la flota (100+ robots).
- Entrenamiento de modelos de navegación y visión por computadora.
- Optimización de rutas con datos históricos de la ciudad.
- Mantenimiento predictivo de la flota completa.
- Dashboard centralizado para operaciones en tiempo real.

5. LABORATORIO PBL: PIPELINE COMPLETO DE CADENA DE FRÍO

LAB 1 — Pipeline de Datos IoT para Cadena de Frío

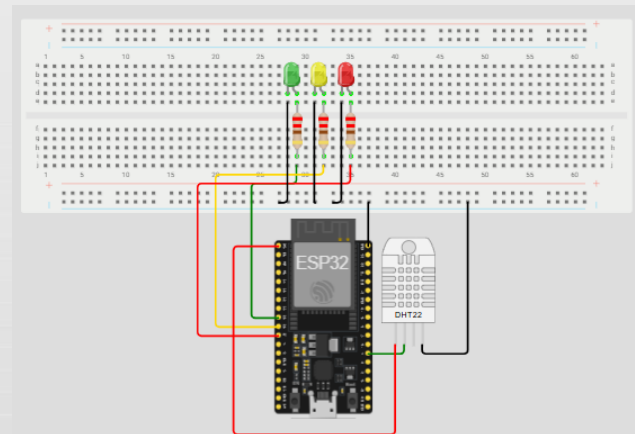
🔥 INDIVIDUAL— 45 MIN EN CLASE

📄 ESCENARIO: Una empresa de logística detecta pérdidas por rupturas de cadena de frío no registradas

Los camiones refrigerados transportan productos farmacéuticos que deben mantenerse entre -2°C y 8°C . Se han detectado pérdidas de producto por rupturas de la cadena de frío que nadie registró. Su equipo debe diseñar e implementar un pipeline de datos IoT completo que capture, transmita, almacene, analice y visualice las lecturas de temperatura de cada camión, y que genere alertas automáticas cuando la temperatura exceda el rango permitido.

🔧 Entregables del LAB

1. Diagrama del pipeline (sensor → broker → BD → dashboard).
2. Código Wokwi (ESP32 + DHT22) simulando camión refrigerado.
3. Código Python (Colab) que analice los datos generados.
4. Regla de alerta: lógica de ruptura de cadena de frío.



Arduino IDE · Código Parte 1 – Configuración



Sección de configuración

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

// == CONFIGURACIÓN DEL HARDWARE ==
#define DHTPIN 4 // GPIO4 → pin DATA del DHT22
#define DHTTYPE DHT22 // Sensor de temp/humedad
#define LED_OK 25 // LED verde: cadena OK
#define LED_ALERT 26 // LED rojo: ruptura detectada

// == CONFIGURACIÓN DE RED ==
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// == CONFIGURACIÓN MQTT ==
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* topic_data = "iot-fpuna/sem3/frío/data";
const char* topic_alert = "iot-fpuna/sem3/frío/alertas";

// == DATOS DEL CAMIÓN ==
const char* camion_id = "CAM-PY-001";
const char* ruta = "Asuncion-CDE";
const float TEMP_MIN = -2.0; // Límite inferior °C
const float TEMP_MAX = 8.0; // Límite superior °C

// == OBJETOS GLOBALES ==
WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;
int alertas_consecutivas = 0;
```

Explicación

`#include <WiFi.h>`

Librería nativa del ESP32 para manejar conexiones WiFi.

`#include <PubSubClient.h>`

Cliente MQTT para Arduino. Se instala desde Libraries en Wokwi.

`#include <DHT.h>`

Librería para sensores DHT11/DHT22. Leer temp y humedad.

`TEMP_MIN / TEMP_MAX`

Rango permitido de la cadena de frío farmacéutica: -2°C a 8°C según OMS.

Dos topics MQTT:

topic_data: telemetría normal cada 5s.
topic_alert: solo cuando hay ruptura de cadena de frío.

Arduino IDE · Código Parte 2 — setup() y conexiones



Funciones de conexión WiFi, MQTT y setup()

```
void conectarWiFi() {
  Serial.print("Conectando WiFi");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println(" OK - IP: " +
  WiFi.localIP().toString());
}

void conectarMQTT() {
  while (!mqtt.connected()) {
    Serial.print("Conectando MQTT...");
    if (mqtt.connect(camion_id)) {
      Serial.println(" Conectado al broker!");
    } else {
      Serial.print(" Error: ");
      Serial.println(mqtt.state());
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  // Inicializar hardware

  dht.begin();
  pinMode(LED_OK, OUTPUT);
  pinMode(LED_ALERT, OUTPUT);

  // Conexiones
  conectarWiFi();
  mqtt.setServer(mqtt_server, mqtt_port);

  Serial.println("=== CAMIÓN REFRIGERADO IoT ===");
  Serial.println("Ruta: " + String(ruta));
}
```

Explicación

WiFi.begin(ssid, pass)

Inicia conexión WiFi. En Wokwi usa la red virtual "Wokwi-GUEST" sin contraseña.

mqtt.connect(camion_id)

Se conecta al broker usando el ID del camión como client_id. Debe ser único por dispositivo.

mqtt.state()

Devuelve código de error MQTT:

- 4 = timeout
- 2 = red caída
- 1 = rechazado
- 0 = conectado

pinMode(LED, OUTPUT)

Configura GPIO como salida digital para controlar LEDs indicadores.

Arduino IDE · Código Parte 3 — loop() y lógica de alerta



Loop principal: lectura, alerta, publicación JSON

```
void loop() {
  if (!mqtt.connected()) conectarMQTT();
  mqtt.loop();

  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) { delay(2000); return; }

  seq++;

  // == LÓGICA DE ALERTA ==
  bool ruptura = (t > TEMP_MAX || t < TEMP_MIN);

  if (ruptura) {
    alertas_consecutivas++;
    digitalWrite(LED_ALERT, HIGH); // LED rojo ON
    digitalWrite(LED_OK, LOW);
  } else {
    alertas_consecutivas = 0;
    digitalWrite(LED_ALERT, LOW);
    digitalWrite(LED_OK, HIGH); // LED verde ON
  }

  // == CONSTRUIR JSON ==
  char json[300];
  snprintf(json, sizeof(json),
    "{\"camion_id\":\"%s\",",
    "\"ruta\":\"%s\",",
    "\"temp_c\":%.1f,",
    "\"humedad_pct\":%.1f,",
    "\"estado\":\"%s\",",
    "\"alertas_seg\":\"%d\",",
    "\"seq\":\"%d\",",
    "camion_id, ruta, t, h,",
    "ruptura ? \"RUPTURA\" : \"OK\",",
    "alertas_consecutivas, seq);

  // == PUBLICAR ==
  mqtt.publish(topic_data, json);
  Serial.println(json);

  // Alerta en topic separado si 3+ lecturas fuera de
  rango
  if (alertas_consecutivas >= 3) {
    mqtt.publish(topic_alert, json);
    Serial.println("*** ALERTA CRITICA PUBLICADA ***");
  }

  delay(5000); // Leer cada 5 segundos
}
```

Explicación

Lógica de alerta:

Si temp > 8°C o < -2°C → ruptura = true.
Enciende LED rojo.

alertas_consecutivas

Contador que se incrementa si la ruptura persiste. Se resetea a 0 cuando vuelve al rango normal. Esto evita falsas alarmas por una sola lectura errónea.

Regla de 3 lecturas:

Solo publica en topic_alert si hay 3 o más lecturas consecutivas fuera de rango (15 segundos). Esto filtra ruido y picos transitorios.

snprintf(json, sizeof...)

Formatea el JSON de forma segura.
sizeof(json) previene desbordamiento del buffer de 300 bytes.

¿Qué es Google Colab? – Laboratorio de Datos

Google Colaboratory (Colab) es un entorno gratuito de notebooks Python que corre en la nube de Google. No requiere instalar nada: solo necesitás un navegador y una cuenta de Google. Viene con todas las librerías de ciencia de datos preinstaladas.



Paso a Paso – Cómo Usar Google Colab

Abrir Colab

Ir a colab.research.google.com
Iniciar sesión con cuenta Google.
Click en "Nuevo Cuaderno".

Crear celda de código

Click en "+ Código" arriba.
Escribir código Python.
Presionar Shift+Enter para ejecutar.

Instalar librerías (si falta)

Escribir en una celda:
`!pip install nombre_libreria`
pandas, numpy y matplotlib
ya vienen preinstaladas.

Subir archivo CSV

Click en icono de carpeta (izq).
Click en "Subir" (flecha arriba).
Seleccionar el CSV desde tu PC.

Cargar datos

```
import pandas as pd
df = pd.read_csv('archivo.csv')
df.head() # Ver primeras 5 filas
```

Exportar notebook

Archivo → Descargar → .ipynb
Este archivo es el que suben
a la plataforma EDUCA.

¿Qué es pandas? — La Herramienta Clave para Datos



pandas en 1 minuto

pandas es una librería de Python para análisis y manipulación de datos tabulares. Fue creada en 2008 por Wes McKinney mientras trabajaba en finanzas. El nombre viene de "Panel Data" (datos en panel), un término de econometría.

Su estructura central es el **DataFrame**: una tabla con filas y columnas, como una hoja de cálculo pero programable con Python. Cada columna puede ser de un tipo diferente (números, texto, fechas).

En IoT usamos pandas para: cargar datos CSV de sensores, calcular estadísticas, filtrar anomalías, agrupar por sensor/fecha y preparar datos para gráficos.



Analogía: pandas = Excel con superpoderes

Pensá en pandas como Google Sheets pero dentro de Python:

- Una hoja = un DataFrame
- Una columna = una Series
- Una fila = un registro (lectura del sensor)
- Fórmulas = funciones como `.mean()`, `.std()`
- Filtros = condiciones como `df[df['temp'] > 30]`

La diferencia: pandas maneja millones de filas sin problema y se automatiza con código.



Comandos pandas Más Usados en IoT

Comando	Qué Hace	Ejemplo
<code>pd.read_csv()</code>	Cargar archivo CSV como DataFrame	<code>df = pd.read_csv('datos.csv')</code>
<code>df.head()</code>	Ver las primeras 5 filas	<code>df.head(10)</code> → primeras 10
<code>df.describe()</code>	Estadísticas: media, std, min, max, percentiles	<code>df['temp'].describe()</code>
<code>df.groupby()</code>	Agrupar por columna y calcular	<code>df.groupby('sensor').mean()</code>
<code>df[condición]</code>	Filtrar filas que cumplen condición	<code>df[df['temp'] > 30]</code>
<code>df.plot()</code>	Gráfico rápido integrado con matplotlib	<code>df['temp'].plot(kind='line')</code>

LAB 1 – Guía de Desarrollo Paso a Paso

1 Diseñar la arquitectura

Dibujar diagrama del pipeline completo: sensor DHT22 en camión → ESP32 → WiFi/4G → broker MQTT → base de datos (elegir cuál) → dashboard. Usar Draw.io o papel.

2 Implementar en Wokwi

Adaptar código del ejercicio anterior. Simular camión: temp entre -2°C y 8°C con anomalías. Agregar campo "camion_id" y "ruta" al JSON. Publicar en topic propio.

3 Analizar con Python

En Google Colab: crear CSV simulado con 100+ lecturas. Usar pandas para calcular estadísticas. Graficar serie temporal con matplotlib. Marcar anomalías.

4 Definir regla de alerta

Establecer lógica: si temp > 8°C o temp < -2°C durante más de 3 lecturas consecutivas → alerta de ruptura de cadena de frío.

LAB 1 • Código Python — Análisis de Cadena de Frío



Código Completo — Copiar en Google Colab

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Generar datos simulados de camión refrigerado
np.random.seed(42)
n = 120 # 120 lecturas (cada 5 min = 10 horas)
tiempo = pd.date_range("2026-04-28 06:00",
                        periods=n, freq="5min")
temp_normal = np.random.normal(3.0, 1.5, n)
# Inyectar 2 anomalías (rupturas de cadena de frío)
temp_normal[40:45] = np.random.uniform(10, 14, 5)
temp_normal[90:93] = np.random.uniform(-5, -3, 3)

df = pd.DataFrame({"timestamp": tiempo,
                   "camion_id": "CAM-PY-001",
                   "temp_c": np.round(temp_normal, 1)})

# 2. Estadísticas descriptivas
print(df["temp_c"].describe())

# 3. Detectar anomalías (fuera de rango -2 a 8)
df["alerta"] = (df["temp_c"] > 8) | (df["temp_c"] < -2)
anomalias = df[df["alerta"]]
print(f"Alertas detectadas: {len(anomalias)}")

# 4. Graficar con zonas de alerta
plt.figure(figsize=(12, 4))
plt.plot(df["timestamp"], df["temp_c"],
         color="#00ACC1", linewidth=1.2)
plt.axhline(y=8, color="red", linestyle="--",
            label="Límite sup (8°C)")
plt.axhline(y=-2, color="blue", linestyle="--",
            label="Límite inf (-2°C)")
plt.fill_between(df["timestamp"], -2, 8,
                 alpha=0.08, color="green")
plt.scatter(anomalias["timestamp"],
            anomalias["temp_c"], color="red",
            s=60, zorder=5, label="Anomalías")
plt.xlabel("Tiempo"); plt.ylabel("Temp (°C)")
plt.title("Cadena de Frío - CAM-PY-001")
plt.legend(); plt.tight_layout(); plt.show()
```

Explicación Clave

`np.random.normal(3.0, 1.5)`

Genera datos normales centrados en 3°C (rango válido). Simula un camión que funciona bien.

Inyección de anomalías:

Líneas 9-10: simula una puerta abierta (temp sube a 10-14°C durante 25 min).

Línea 11: simula falla del compresor (temp baja a -5°C).

Regla de alerta:

`df["alerta"] = (temp > 8) | (temp < -2)`

Marca True cada lectura fuera de rango. Permite contar y localizar cada ruptura.

Visualización:

`fill_between` marca la zona segura en verde.
`scatter` resalta anomalías en rojo.

LAB 2 – Tarea para EDUCA: Análisis Completo con Python



TAREA INDIVIDUAL



ESCENARIO: Un gerente de planta necesita un reporte de analítica sobre 3 sensores de temperatura durante 7 días

El dataset CSV (proporcionado en EDUCA) contiene lecturas cada 5 minutos de 3 sensores industriales. El gerente necesita saber: ¿cuál sensor tiene mayor variabilidad? ¿se detectan patrones diurnos? ¿hay anomalías que indiquen fallas? Usted debe responder con datos y gráficos profesionales.



Requisitos Técnicos

- 1. Cargar CSV en Google Colab con pandas.
- 2. Calcular: media, mediana, std, mín, máx, P5, P95 por sensor.
- 3. Gráfico de línea temporal con 3 sensores superpuestos.
- 4. Histograma de distribución para 1 sensor a elección.
- 5. Detectar anomalías con Z-score > 3 y marcar en gráfico.
- 6. Párrafo de conclusiones interpretando resultados.

Tarea EDUCA – Paso a Paso Detallado

Descargar y cargar CSV

5 min

```
Descargar sensor_data_7dias.csv desde EDUCA.  
Subir a Google Colab (icono carpeta → Subir).  
Ejecutar:  
import pandas as pd  
df = pd.read_csv('sensor_data_7dias.csv')  
df.head()
```

Explorar el dataset

5 min

```
df.shape → filas y columnas.  
df.dtypes → tipos de datos.  
df['sensor_id'].unique() → nombres de los 3  
sensores.  
df.info() → resumen completo.
```

Estadísticas descriptivas

10 min

```
stats =  
df.groupby('sensor_id')['temp_c'].describe()  
stats['P5'] =  
df.groupby('sensor_id')['temp_c'].quantile(0.  
05)  
stats['P95'] =  
df.groupby('sensor_id')['temp_c'].quantile(0.  
95)  
print(stats)
```

Gráfico de línea temporal

10 min

```
for sensor in df['sensor_id'].unique():  
    subset = df[df['sensor_id']==sensor]  
    plt.plot(subset['timestamp'],  
subset['temp_c'], label=sensor)  
plt.legend()  
plt.title('Temperatura 7 días')
```

Histograma de distribución

5 min

```
sensor1 = df[df['sensor_id']=='SENSOR-  
A']['temp_c']  
plt.hist(sensor1, bins=30, color='#00ACC1',  
alpha=0.7)  
plt.title('Distribución SENSOR-A')  
plt.xlabel('Temperatura °C')
```

Detectar anomalías (Z-score)

10 min

```
from scipy import stats  
df['zscore'] =  
df.groupby('sensor_id')['temp_c'].transform(  
lambda x: stats.zscore(x))  
anomalias = df[df['zscore'].abs() > 3]  
print(f'Anomalías: {len(anomalias)}')  
# Marcar en gráfico con plt.scatter()
```

Tarea EDUCA – Conclusiones y Entrega

Guía para las Conclusiones

Responder estas 4 preguntas del gerente:

1. ¿Cuál sensor tiene mayor variabilidad?

Comparar la desviación estándar (std) de cada sensor. El de mayor std es el más variable. Mencionar el valor numérico.

2. ¿Se detectan patrones diurnos?

Observar en el gráfico de línea temporal si la temp sube de día y baja de noche. Describir el ciclo.

3. ¿Hay anomalías que indiquen fallas?

Reportar cuántas anomalías $Z > 3$ se encontraron, en qué sensor(es) y en qué momentos.

4. ¿Qué recomienda al gerente?

Sugerir acción concreta: revisar sensor X, calibrar equipo Y, implementar alerta automática.

Entregables Finales

A) Notebook .ipynb exportado

Archivo → Descargar → .ipynb. Verificar que todas las celdas estén ejecutadas y los gráficos visibles.

B) Tabla de estadísticas

Puede ser la salida de `df.describe()` directamente en el notebook, o una captura de pantalla.

C) Gráficos (3 mínimo)

Línea temporal, histograma y anomalías marcadas. Deben tener título, ejes etiquetados y leyenda.

D) Párrafo de conclusiones

Mínimo 200 palabras respondiendo las 4 preguntas del gerente con evidencia de los datos.

Dataset CSV: sensor_data_7dias.csv – Disponible en EDUCA

El CSV contiene 6.048 filas (3 sensores $\times$ 2.016 lecturas cada 5 min $\times$ 7 días). Columnas: timestamp, sensor_id (SENSOR-A, SENSOR-B, SENSOR-C), temp_c, humedad_pct. Incluye anomalías inyectadas para que las detecten.

6. RESUMEN, BIBLIOGRAFÍA Y PRÓXIMOS PASOS

Resumen — Ideas Clave de la Semana 3



4 IDEAS CLAVE QUE DEBES LLEVAR DE HOY

1

Los datos son el recurso central de IoT — y el 95% nunca se analizan

Sin una estrategia de gestión (pipeline), los sensores generan ruido, no valor. Series temporales, eventos y estados requieren tratamiento diferenciado. La cadena Dato → Información → Acción es lo que transforma IoT en resultado de negocio.

2

El pipeline de datos es la columna vertebral de toda solución IoT

Las 5 etapas (ingesta, almacenamiento, procesamiento, análisis, visualización) deben diseñarse como un sistema integrado. Batch para reportes e historial; stream para alertas y control. La elección entre edge y cloud depende del caso.

3

La analítica IoT avanza en niveles de sofisticación y valor

Descriptiva (¿qué pasó?) → Diagnóstica (¿por qué?) → Predictiva (¿qué pasará?) → Prescriptiva (¿qué hacer?). Machine Learning habilita los niveles superiores con algoritmos que aprenden de datos históricos.

4

Casos reales demuestran que estos conceptos operan a escala industrial

Artemis (NASA): telemetría en tiempo real con 1.000+ sensores y detección de anomalías en ms. Robots repartidores: arquitectura híbrida edge + cloud con ~500 MB/hora por robot. El pipeline que diseñamos en el LAB sigue los mismos principios.



PRÓXIMOS PASOS: Semana 4: Smart Manufacturing e IIoT — Monitoreo de activos, mantenimiento predictivo y KPIs

Referencias



Bibliografía — Semana 3: Gestión de Datos, Analítica y ML en IoT

Textos Base

Bahga, A. & Madiseti, V. (2014). Internet of Things: A Hands-On Approach. Capítulos de gestión de datos y cloud.

Buyya, R. & Dastjerdi, A.V. (eds.) (2016). Internet of Things: Principles and Paradigms. Cap. procesamiento de datos.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly. Cap. 1-3.

Goodfellow, I. et al. (2016). Deep Learning. MIT Press. Cap. 1 (opcional).

Casos de Éxito y Herramientas

NASA Artemis Program. Orion Spacecraft Telemetry. nasa.gov/artemis.

Starship Technologies. Autonomous Delivery Robot Overview. starship.xyz.

InfluxData. Time Series Database Concepts. docs.influxdata.com.

Eclipse Mosquitto. MQTT Broker. mosquitto.org.

Documentación pandas, matplotlib, scikit-learn. Plataforma EDUCA.



¡MUCHAS GRACIAS!